Author Response to Reviews of

DAENet: Making Strong Anonymity Scale in a Fully Decentralized Network

Tianxiang Shen, Jianyu Jiang, Yunpeng Jiang, Xusheng Chen, Ji Qi, Shixiong Zhao, Fengwei Zhang, Xiapu Luo, Heming Cui

IEEE Transactions on Dependable and Secure Computing,
`Manuscript ID:TDSC-2019-11-0652`

Dear Referees,

Thank you very much for your valuable comments. We are profoundly grateful for the feedback, which greatly helped us improve the quality of the origin manuscript. Accordingly, we have modified the manuscript by taking into consideration both reviewers' requests. We take it as a positive gesture that both referees have submitted positive remarks about the manuscript. The revised manuscript has been re-written by taking into account all these recommendations. The changes in the revised manuscript are catalogued here. We have, thus, replied to the Reviewers' comments, and indicated clearly what we have changed.

RC:*Reviewer Comment*, AR:*Author Response*, □ Manuscript text

1 Associate Editor

RC: Please carefully address every comment from each reviewer.

AR: Thank you for allowing us the chance to improve our work through the revised manuscript.We have tried to include all the suggested changes in the updated manuscript. Individual reviewer-specific comments, which have been incorporated in the updated manuscript, are specifically catalogued in this document.

2 Reviewer #1

RC: I like the idea of the paper, but I just don't think it's publishable in the current format. The writing, organization, and evaluation need to be significantly improved. It looks like you used the Loopix paper as a guideline for your evaluation, but that's a very different network, and so your eval is going to be different.

AR: Thank you for your like of our ideas and pointing out the shortcomings of our original manuscript. For writing, we have corrected the spelling and grammar mistakes, and given the revised work a thorough read-through to reduce writing errors further. For paper organization, we have re-written the main part of this paper to highlight the contribution of this work better and avoid repetition and unnecessary section pointers. For evaluation, we have re-written the evaluation setup to clearly introduce our motivation of the usage of baselines systems such as Loopix. The following is part of the updated description of our evaluation setup that gives explanation of choosing the baseline systems.

We compared DAENet's performance with two state-of-art shuffle-based anonymous communication systems: Loopix and Dissent. Loopix is a popular open-sourced anonymous network that leverages poisson-mixing shuffle strategy to protect users in the same conversation from being observed by global passive attackers, which is also guaranteed by DAENet and has been proved in §5.1.1. We also compared DAENet's performance with Dissent. Dissent is another open-sourced anonymous network that leverages verifiable shuffles to defend against global passive attacks. Although Dissent suffers from long-term active intersection attacks, it is well-known for its support of low-latency communications compared to other shuffle-based systems (e.g., Riposte, Atom). Other shuffle-based systems such as Karaoke and Vuvuzela are not evaluated because they are not open-sourced.

RC: English grammar problems - The Internet, not "Internet" and etc. There are an *enormous* number of errors in this paper. Misused terms like "off band" and "out of bounds" instead of "out-of-band", spelling errors, typos, capitalization errors. You should run this through grammarly at the least.

RC: In line 38 you should probably use peer-to-peer, rather than p2p, or at least introduce it then switch to the abbreviation. Also, it should be written "P2P" rather than lowercase.

RC: 4.1 line 35 You mean out-of-band, not off-band. There are quite a few errors like this.

AR: Thank you for your advices. We apologize for the grammar problems, misused terms and other writing mistakes appeared in the original manuscript. We have, thus, given the revised work a thorough read-through to reduce such writing errors further.

RC: The discussion of Tor vulnerabilities in the introduction is dated (based on a paper from 2005), and does not take into account more recent Tor improvements, such as cover traffic, guard nodes, and etc.

AR: We have moved the detailed description of Tor into our new "Discussion" chapter, and discussed Tor's state-of-art vulnerabilities and defense techniques (e.g., cover traffic and guard node selection). We have included the following information in Section 8 of the revised manuscript.

The most popular relay-based anonymous communication system is Tor [15]. Due to its popularity and transparent development processes [61], many researchers have explored attacks that can de-anonymize Tor users and hidden-service providers by monitoring the network traffic. Recent attack directions for Tor include website fingerprinting [62], [63], [64], [65], traffic correlation [10], [11], [12], [66], congestion attack [67], [68] and targeted DoS [39], [69], [70]. Meanwhile, researchers also propose methods to enhance Tor's security by optimizing the bandwidth report for selecting guard nodes [71] and monitoring circuit construction [72]. Also, some recent Tor improvements consider generating cover traffic within middle routers of circuits, such that the middle routers can hide any relationship between compromised entry and exit nodes [73], [74].

RC: How/who assigns ID's? (section 2.2) You discuss this later, but not here.

AR: We have changed the paper organization and given more details in DAENet's underlying communication network (i.e., Chord). Specifically, in the underlying network, each participant joins the network by sending a *join* request to a known DAENet node. The node will assign an identifier to the participant by using the consistent hashing technique and help the participant set up its routing table by several rounds of consultations to its neighbors. In DAENet, the node is called the *guarder node*. The guarder node also serves as an attestation server to verify whether an unmodified DAENet's program is executed inside a real SGX host. If the node passes the attestation, the guarder node replies with an automatically generated identifier, which indicates the node's location in the DAENet network; If the participant fails the attestation, it will be prohibited from joining the network.

RC: DAE v.s. SGX-Tor – I think this part is in the wrong place. The comparison vs SGX-Tor makes a lot of claims that haven't been supported or even explained. Possibly move this later in the paper.

AR: Thank you for pointing out the organization problem. We have moved this comparison to related work.

RC: I don't think it's so easy to just say "SGX is trusted" when there are several known vulnerabilities that compromise the security of the system. For SGX-Tor, if SGX fails, you're left with Tor, which is somewhat secure. With your system, if SGX is compromised, there is no security left at all. This is a major difference, and I think it merits at least a discussion of the impact of known SGX flaws on your system.

AR: Yes, we cannot easily trust the features provided by Intel SGX as SGX is susceptible to other attacks, such as the side-channel attacks. In our work, it is important to enclose all private operations (e.g., fully random messaging to neighbors) and sensitive information (e.g., identifers of communication parties) inside the enclave to make sure that it will not be exposed to untrusted participants. As SGX is susceptible to side-channel attacks where malicious software on the same platform can infer enclave data access patterns by monitoring shared resources such as caches, it is fixable by using well-known Oblivious Ram (ORAM) algorithms, such as ZeroTrace. We have added a discussion of SGX flaws in section 7 of the revised manuscript.

Current DAENet implementation do not integrate side-channel attack defenses. As SGX is susceptible to side-channel attacks where malicious software on the same platform can infer enclave data access patterns by monitoring shared resources such as caches [53], [54], it is fixable by using well-known Oblivious Ram (ORAM) algorithms, such as ZeroTrace [55].

RC: You discuss the different attacks you will protect against, but you don't specify *how* the system will protect against them. Contrast this with Section 3.3 of the SGX-Tor paper to see what I'm talking about.

RC: Organization – the organization of the paper makes it difficult to read. For example, you often refer to things you haven't discussed yet by saying "see section XXX". However, often this is critical information to understanding the thing you are currently discussing. Skipping around in the paper is hard on the reader, and makes it difficult to evaluate what you've done. This suggests that you may need

to reorganize your paper to more logically present your work. There is also a large amount of repetition in the paper. For example in 3.2 line 54 you talk about dead drop nodes, then 3.3 you discuss them again, and repeat most of 3.2 and then add a bit to the discussion.

AR: Thank you for your valuable comments. In the third chapter where we talked about the security goals before, we have added corresponding defense techniques for clearer elaboration. Specifically, there are three security goals and corresponding privacy approaches. First, to defend against global passive attacks, we shuffle real messages with indistinguishable dummy messages, and disseminate messages to each participant's neighbors with full randomness to mess up the orders of input and output packets. Second, to defend against eclipse attacks, we use close-loop packet to detect malicious packet drops by verifying each on-path relay's *Proof-of-Forwarding* generated inside SGX. Last, to defend against traceback attacks, we use randomly generated dead drop nodes to build up different and unpredictable routing circuits in each communication round, preventing active attackers tracking and revealing sender identities.
For the organization, we have completely re-organized the flow and the structure of the paper. We have re-written the introduction to help readers understand the position of DAENet better, re-written the background to give more details of anonymous communication network and the underlying P2P structure, and re-written the overview to correlate every attack and defense techniques. In addition, all chapters are updated with a thorough read-through to avoid mistakes such as undefined terms, misleading section pointers and etc.

RC: The theorems in this section aren't actually theorems. They're just statements of equations that come directly from your formulations. Also, you refer to Theorem 2 as a "lemma" in the text preceding it, which is not a theorem. You also say it's an "upper bound" but there is no inequality, which an upper bound would imply. You later state that 1/y is the is "the upper bound for an adversary to correctly link the input message x and the corresponding output message x1." which might be true, but this is not derived or demonstrated in your paper. This looks like what they did in the Loopix paper, but they provided proofs in an appendix, which you do not.

AR: Thank you for pointing out the writing mistake in this paper. The previous Theorem 1 that described the process of pulling messages from shuffle pools has been refactored to be a statement in the revised manuscript. The statement is included in section 4.2:

> Denote $\mathcal{N}$ as the total number of participants in the network and k as the number of empty shuffle pools of Alice's all successors. Derived from previous statements, Alice pulls messages from $\log\mathcal{N} - k$ shuffle pools in each protocol run. The pulling process takes the form of Binomial distribution $\mathcal{X} \sim B(\log\mathcal{N}, \alpha)$ where the discrete probability α is the *expected shuffle rate*. In each round, the expected total number of real application messages and dummy messages for Alice to send is $\alpha(\log\mathcal{N} - k)$ and $k + (1 - \alpha)(\log\mathcal{N} - k)$, respectively.

The previous Theorem 2 has been refactored to a Claim which describes the probability of correctly linking an arriving message with an outgoing message in a host. We give an upper bound probability 1/y because the outgoing message must be from the host's shuffle pool and thus is limited to the range of existing messages in current host. As there are totally y messages as we defined, the upper

bound on the probability that adversaries correctly do the traffic correlation is thus 1/y. This upper bound inference applies to all shuffled-based systems that defends against traffic correlation attacks. Loopix handles traffic correlation attack and also gives an upper bound on the attack probability (i.e., 1/(l+k)) because Loopix assumes a shuffle pool is mixed l+k messages presently.

RC: The malicious host detection only seems to work if *all* network packets are dropped. What if the host is only dropping a fraction of messages? It's also unclear how you form the path such that you know who all the participants in the loop path will be, given that you may not be connected to them?

AR: Yes, if a malicious host drops all network packets that pass it through, our protocol will detect it because the malicious host cannot send back a correct *Proof-of-Forwarding*. If a host only drops a fraction of messages, chances are that our *loop* packets may not be dropped and the malicious host will not be detected. However, our protocol still works in this scenario with large probability because all participants periodically send *loop* packets to detect malicious packet drop, such that a malicious host contains at least $\log \mathcal{N}$ *loop* packets from all its predecessors. If one of the *loop* packets are dropped, the malicious host will be detected.
Regarding the relays on the loop path, they are deterministically selected with reference to a random bit b_s generated within enclaves: each relay on the path selects its b_sth neighbor as the next hop. Thus, a participant can request for all the on-path relays by asking its neighbors iteratively.

RC: I don't think I've ever seen the word phishing used in this context.

AR: We have changed the name of the attack on revealing sender identities in the revised manuscript from "phishing attack" to "traceback attack", in order to avoid misunderstandings of the original phishing attacks used in other security domains (e.g., web browsing). In section 4.4, we have added more description of the traceback attack and enhanced the relationship between paragraphs.

RC: One malicious dead drop node won't compromise sender/receiver confidentiality, but what about multiple nodes. Specifically, what fraction of the dead drop nodes need to be non-malicious? This is a key number because it's the easiest place to compromise the network.

AR: Thank you for your valuable comments. In DAENet, the randomly selected dead drop nodes serve as different destinations in each communication round and thus the routing circuit between a sender and receiver is dynamic and unpredictable. If one dead drop node is malicious, the only thing it can do is refuse to exchange the payload and forward packets. If so, the participant can exchange the payload by sending to another unused dead drop nodes in next communication round without compromising anonymity. A malicious dead drop node cannot correlate sender/receiver because the input and output packet order is totally mixed by shuffles. This guarantee still exists even if multiple dead drop nodes are malicious because every dead drop node receives messages from different routing circuits, such that the malicious dead drop nodes cannot correlate a predecessor or successor on the path. We have included the following paragraph to discuss dead drop node failures in section 4.4 of the revised manuscript:

Note that compromised dead drop nodes could fail to execute the payload exchange and claim to be temporarily offline. DAENet tolerates dead drop node failures with a *switch* strategy. The core idea of the strategy is that session nodes do *not* need to wait for a successfully exchanged reply from dead drop nodes in each communication round. If Alice's message m was not sent back by dead drop node N_i, Alice can resend m by switching to another *unused* dead drop node N_j with reference to $DeadDrop_key_j$. DAENet provides such flexibility because DAENet supports reliable datagram transfer, rather than online streaming that needs ordered network packets. Thus, we assume participants can tolerate a reasonable delay of some messages and transfer other packets first when a portion of dead drop nodes fail. In the worst case when all dead drop nodes mapped from $DeadDrop_keys$ are compromised, no successful payload exchange will take place. Since the dead drop nodes are randomly selected, the failure of all dead drop nodes indicates a potential monitoring of the network. Such vulnerability will be quickly detected and participants are suggested to carry on their conversations later.

3 Reviewer #2

With gratitude,
Tianxiang shen
Author of the manuscript